

EARIN Project - Final Solution Report

Project 15: *Heart attack prediction*

By: *Luck Poyet*

1. Introduction

The goal of this project was to implement and compare the performances of three different machine-learning models over a **Binary Classification Problem**: predicting whether a patient is prone to having a heart disease based on a specific set of features.

This **EARIN Project Final Report** first summarizes the conclusions & results from the *Preliminary & Midterm reports*, before showcasing the final, complete solution to the project.

2. Dataset Description

As advised in the instructions, the Dataset is being fetched from the *Kaggle* platform. While the platform offers a wide array of *heart attack prediction* related datasets, it is the **UCI Heart Disease (combined)** that has been chosen.

The combined **UCI Heart Disease** dataset aggregates data from Cleveland, Long Beach, Switzerland and Hungary. This dataset is a well-known go-to for such exercises as it provides a multitude of real clinical observations as well as some geographic variance. While the Cleveland subsection of the UCI Heart Disease bundle was the legacy, most complete option, we decided to use the combined version to add some diversity to the overall dataset, increasing the number of records **from 303 to 920**. Finally, the dataset's concise dozen features shall enable a highly relevant feature importance interpretation during the results analysis, compared to other options that might have a greater number of vague, less impactful features.

3. Models Implemented

The **combined UCI Heart Disease** problem can be solved with a wide array of machine-learning models, such as Logistic Regression, K-Nearest Neighbors, Naive Bayes, Support Vector Machines, or ensemble methods like Random Forests or Gradient Boosting.

For this project, **3 models** are being implemented: the Logistic Regression (*LR*), Support Vector Machine (*SVM*) and Random Forest Classifier (*RFC*). While they are all common options for such a task, they also have specific advantages, highly relevant in this context.

Logistic Regression (baseline)	Provides highly interpretable output and not prone to overfitting, thus making the perfect <i>baseline</i> for the overall project.
Support Vector Machine	Highly potent for deeply tangled features, but requires precise tuning to avoid overfitting.
Random Forest Classifier	Extremely versatile model and very low overfitting rate. It can however favor certain features and its verdict is impractical to interpret.

In order to accurately evaluate these models, their respective hyperparameters were thoroughly tuned to heuristically determine the most effective set for each.

>> Logistic Regression Model

Parameter	Values	Description
C	0.01 1.0 10	<i>Scales the weight values. A lower value prevents overfitting better but can hinder performance.</i>
class_weight	none or balanced	<i>Either doesn't or does adjust weight values inversely proportional to patient status frequencies (healthy/at risk). Prevents the model from lazily following the overall health tendency.</i>

>> Support Vector Machine : Classifier

Parameter	Values	Description
C	0.5 1.0 5.0	<i>Controls the trade-off twixt accuracy and rule simplicity.</i>
kernel	linear or rbf	<i>Either draw the hyperplane in the standard or in an infinite-dimensional space using the RBF function.</i>
gamma	scale, 0.05, 0.1	<i>The SVM focus range while drawing the hyperplane. A lower gamma value considers the overall shape of the data points while a higher value evaluates every nearby point strictly, creating a more complex rule.</i>
class_weight	none or balanced	<i>Like for the Logistic Regression model, toggling weight scaling to account for the dataset bias.</i>

>> Random Forest Classifier

Parameter	Values	Description
n_estimators	100 200 500	<i>Number of decision sub-trees. A higher number of trees provides a diminishing returns at the expense of computational power.</i>
max_depth	4, 8, 10	<i>Number of branches each estimator is allowed to reach. A higher value enables complex patterns understanding but might introduce overfitting.</i>
min_spl_split	2, 5, 10	<i>Minimal sample size for the algorithm to trigger a split. A lower value allows for a closely fitting data points breakdown, but can quickly lead to overfitting.</i>
min_spl_leaf	4, 6, 10	<i>Minimal sample size for a leaf (a sub-tree's end point) to be. A higher value significantly hinders overfitting but loses precision.</i>
class_weight	balanced or balanced_subsample	<i>Cf previous models. While balanced calculates weights based on the entire dataset, balanced_subsample calculates for each estimator specific weights based on their own bootstrap sample.</i>

These hyperparameters have been conscientiously selected in order to prevent overfitting as the dataset remains quite small.

4. Data Preprocessing

Once the models are defined, the dataset itself has to be prepared for seamless data processing. The *Combined UCI Heart Disease* dataset is especially challenging as it includes a **significant number of missing fields**, which is something that must be accounted for.

Eventually, the preprocessing phase was implemented as the following:

- **numerical features:**
 - First completing the potential missing values by **Median Imputation**, which effectively fills the voids while being less sensitive to outliers than the *mean* version.
 - Then **normalizing** every value so that no lower yet highly relevant values are neglected. To proceed, the preprocessing phase relies on **Z-score** normalization, achieving an overall *mean value* of 0 and a *standard deviation* of 1.
- **categorical features:**
 - First completing the potential missing values by **implementing a dummy Null categorical option**, which prevents as mentioned the models from crashing.
 - Then running OneHotEncoder to transform worded categorical data into **model-processable matrices**.

5. Midterm Solution Insights

As showcased in the **Midterm Project Report**, the *Logistic Regression* model performed well and lived up to expectations.

However, a flaw that stood out was the testing method consistency. In fact, while deemed satisfactory for a single algorithm evaluation, the overall process - *especially output-wise* - was too bloated for the upcoming 2 models addition, let alone their respective hyperparameter sets.

Moreover, by using the *sk-learn* `classification_report` output method, we have been so far evaluating a model performance based on **subjective heuristic analysis**. While this method is suited for us human observers, the program requires a way to objectively compare 2 performances, hence introducing the issue of choosing a fitting metric that would truly quantify a run's quality.

Finally, in order to ensure a thorough exploitation of the training dataset, this first implementation attempt missed a critical component: the **4-fold Cross-Validation** method.

Therefore, the whole training & testing process pipeline was **refactored** in order to fully exploit each model's potential, while keeping a relevant, human-readable output display.

6. Final Results Analysis

6.1. Hardened Testing Method

>> Hyperparameters Culling:

As printing and plotting every models' hyperparameter sets would be highly inefficient for conclusion, a culling protocol has been implemented. By selecting the most suited *hyperparameter set* for a model beforehand, the final model-to-model comparative can be significantly more synthetic thus human-readable.

Using the *sk-learn* GridSearchCV function, the program now cross-compares a model's every hyperparameter values at once, each time applying the **4-fold Cross-Validation**. In order to perform the *objective single-metric optimization* aforementioned, a new metric is being implemented: the **F2 score**.

Sorting hyper parameter sets' performance based on the *accuracy* only is dangerous in a medical context as it doesn't emphasize the dismal consequences of a False Negative result compared to a benign False Positive. For this project, a more cautious, recall focused approach was arbitrarily adopted, putting the priority on saving lives rather than getting the highest scores. However, such philosophy may introduce a highly disruptive bias as the model could in theory flag everyone as *ill*, bringing recall score to 1 but **losing all relevance** instantly.

It is to resolve this trade-off that the **F2 score** was implemented. By weighting the recall twice as heavy as the precision, this metric enables a **safety-oriented ranking** while providing the mathematical guardrails to penalize indiscriminate false flags.

>> Model Results Comparison:

Once each model's most effective hyperparameter set is determined, their respective confusion matrix may be plotted altogether for an holistic evaluation.

6.2. Hyperparameters Selection Outcome

• Logistic Regression:

```
Training and Tuning: Logistic Regression...
Fitting 4 folds for each of 8 candidates, totalling 32 fits
-> Best Hyperparameters: {'classifier__C': 0.01, 'classifier__class_weight': None}
-> Best 4-FCV Validation F2 Score: 0.8434
```

It is the {class_weight: None ; C: 0.01} set that shew the best results. Such outcome is consistent with the midterm *Logistic Regression* implementation where the same tuning gave the best recall value alongside overall satisfactory results.

- **Support Vector Classifier:**

```
Training and Tuning: Support Vector Machine...
Fitting 4 folds for each of 36 candidates, totalling 144 fits
-> Best Hyperparameters: {'classifier__C': 0.5, 'classifier__class_weight': None, 'classifier__gamma': 0.05, 'classifier__kernel': 'rbf'}
-> Best 4-FCV Validation F2 Score: 0.8509
```

Overall, it seems that the softening hyperparameters (low C and gamma values) provided the best results. This outcome fits the narrative as mentioned already, the small **UCI Heart Disease** dataset makes the models critically prone to overfitting. Thus, by lightly penalizing training errors and not focusing too harshly on every single data point, the *Support Vector Machine* was able to create a rule that efficiently grasped the dataset overall pattern.

- **Random Forest Classifier:**

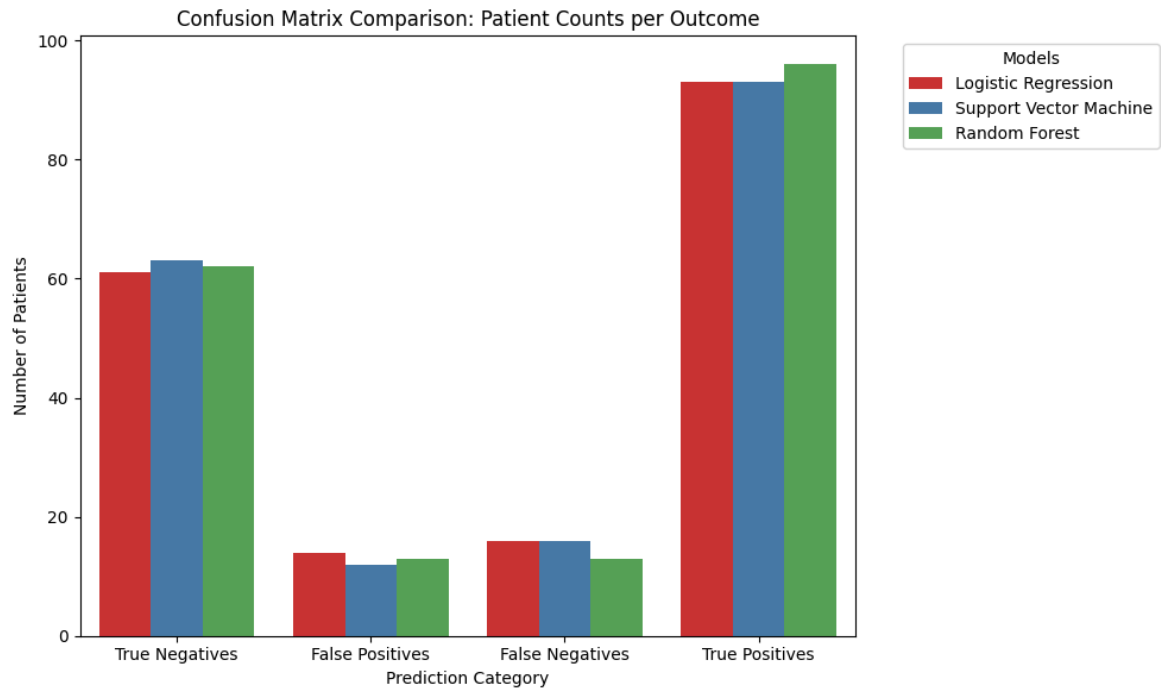
```
Training and Tuning: Random Forest...
Fitting 4 folds for each of 72 candidates, totalling 288 fits
-> Best Hyperparameters: {'classifier__class_weight': None, 'classifier__max_depth': 8, 'classifier__min_samples_leaf': 10, 'classifier__min_samples_split': 2, 'classifier__n_estimators': 200}
-> Best 4-FCV Validation F2 Score: 0.8515
```

Eventually, a thorough behind-the-scene heuristic hyperparameters tuning enabled some highly satisfactory results for the *Random Forest Classifier*, which take the lead with the **highest F2 Score**.

One may notice the high `min_samples_leaf` and `min_samples_split` values (10). These hyperparameters were likely the key to create a **potent, generic rule** as they prevented the algorithm from breaking patterns down too thin, hence preventing overfitting and performance loss.

6.3. Final Models Comparative

- Confusion Matrices Plotting



- Terminal Output Table

```
Final Evaluation on Unseen Test Set
```

Model	Train F2	Test F2	Train Acc	Test Acc	Overfit Gap
Logistic Regression	0.8557	0.8564	0.8288	0.8370	-0.0082
Support Vector Machine	0.8682	0.8595	0.8478	0.8478	0.0000
Random Forest	0.8673	0.8807	0.8451	0.8587	-0.0136

- Results Review

With overfitting rates being either null or negative, we can safely affirm that all 3 models were properly trained, **thoroughly isolating patterns** instead of learning the dataset.

While the results are tightly packed, the most efficient model ends up being the **Random Forest Classifier**, which scored both the highest *accuracy* and *F2 score*, meaning it assigned the correct diagnosis to the most patients while still enforcing on the lowest *False Negatives* number possible.

7. Ablation Study

So far, the models were trained and evaluated without considering the *origin* feature. During the *midterm implementation*, the possible impact of this outstanding feature was questioned: would it introduce **dismal noise** for the models as one patient's geography as it is hasn't any influence on his health, or would it instead act as a **proxy** for elements of influence that aren't evaluated by the study.

To proceed, the whole program has been ran again, this time adding the dataset columns to the CATEGORICAL_FEATURES array. While keeping the previous best performing *hyperparameter sets* could have challenged the model robustness directly, performing a new selection using GridSearchCV was deemed more interesting as it showcases whether the added feature is redundant or not.

7.1. Hyperparameters Adaptation

```
Training and Tuning: Logistic Regression...
Fitting 4 folds for each of 8 candidates, totalling 32 fits
-> Best Hyperparameters: {'classifier__C': 0.01, 'classifier__class_weight': None}
-> Best 4-FCV Validation F2 Score: 0.8411

Training and Tuning: Support Vector Machine...
Fitting 4 folds for each of 36 candidates, totalling 144 fits
-> Best Hyperparameters: {'classifier__C': 0.5, 'classifier__class_weight': None, 'classifier__gamma': 'scale', 'classifier__kernel': 'rbf'}
-> Best 4-FCV Validation F2 Score: 0.8547

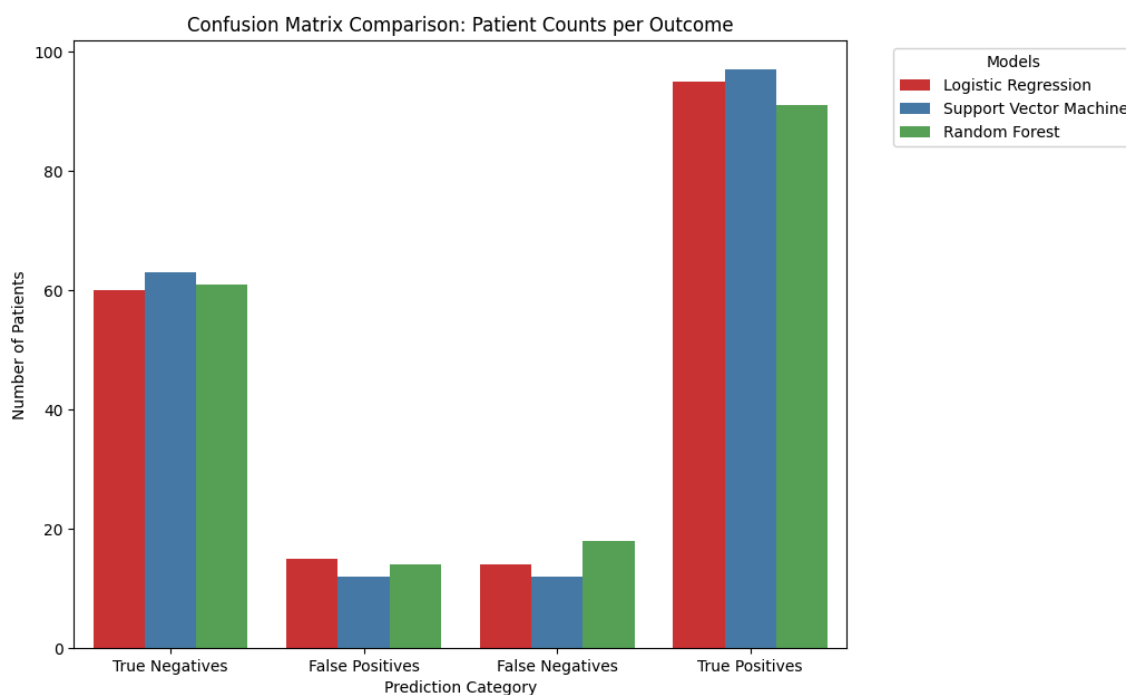
Training and Tuning: Random Forest...
Fitting 4 folds for each of 108 candidates, totalling 432 fits
-> Best Hyperparameters: {'classifier__class_weight': None, 'classifier__max_depth': 8, 'classifier__min_samples_leaf': 10, 'classifier__min_samples_split': 2, 'classifier__n_estimators': 100}
-> Best 4-FCV Validation F2 Score: 0.8420
```

While the *Logistic Regression* model stuck to the same hyperparameters, one may notice that the *Support Vector Machine* swapped its gamma value for scale ($\gamma = \frac{1}{n_{\text{features}} * \text{variance}} \approx 0.089$) thus finely tweaking it from the previous 0.05, and the *Random Forest Classifier* dropped a 100 of estimators.

Moreover, we may already notice some changes regarding the best **F2 scores** so far. While both the *Logistic Regression* and *Support Vector Machine* models kept virtually the same result, the *Random Forest Classifier* lost **almost a percent** compared to the previous evaluation.

7.2. Models Actualized Performances

• Confusion Matrices Plotting



- **Terminal Output Table**

```
Final Evaluation on Unseen Test Set
```

Model	Train F2	Test F2	Train Acc	Test Acc	Overfit Gap
Logistic Regression	0.8528	0.8700	0.8302	0.8424	-0.0122
Support Vector Machine	0.8829	0.8899	0.8587	0.8696	-0.0109
Random Forest	0.8669	0.8410	0.8438	0.8261	0.0177

- **Results Review**

Shockingly, adding the origin feature completely turned the table. The previously best performing *Random Forest Classifier* **lost 4 points of F2 score** and **2.5 points of accuracy**. On the other hand, both the *Logistic Regression* and *Support Vector Machine* models showed significant improvement in the 2 metrics, with the SVM outperforming the other two in every single metrics of the plotted confusion matrices.

8. Conclusion

The project 3 models were successfully implemented with some highly satisfactory results. **Overfitting** was virtually completely mitigated and this despite the small size of the UCI Heart Disease dataset.

Initially, the **Random Forest Classifier** outshone its peers. By dialing down estimator branches through finely constrained hyperparameters, the ensemble model successfully mapped the physiological features pattern.

However, the ablation study, which introduced the origin feature to the dataset, revealed a critical divergence in how the 3 algorithms handle disparate data structures.

Indeed, as decision trees greedily maximize immediate information gain, it is likely that the patient origin was used as early root split by the RFC. Therefore, it somewhat siloed the available data, starving the lower branches from the information density required to properly learn the underlying rule.

Conversely, the **Support Vector Machine** and **Logistic Regression** algorithms greatly improved. As their operating principle relies on assigning global weights to ultimately draw mathematical boundaries, the 2 models successfully incorporated the new feature as an independent bias variable.

This project showcased that a given model robustness not only depends on its hyperparameters tuning, but also on its inherent compatibility with the task. While the decision tree-based algorithm was the most potent for a purely physiological evaluation, the mathematical models proved themselves more effective at assimilating broad, abstract biases.